

v1.0.8

PyCuDAL

From Legacy SAS to Open-Source Python

Parametric acceptance limits for USP <905> Content Uniformity • USP <711> Dissolution

Project Walkthrough — Design, Verification & Validation

AGENDA

What We'll Cover

1

The Problem

Why a decades-old SAS system needed rethinking

2

What PyCuDAL Does

Four modules, three analysis modes, one shared engine

3

Faithful Translation

Verifying every formula against the original SAS macros

4

Performance Engineering

Vectorizing the math for real speed gains

5

Rigor in Practice

The CUSP2 investigation and an independent R cross-check

6

Validation & What's Next

The protocol in progress, and how you can help

BACKGROUND

Why Replace a System That Works?

- CuDAL calculates parametric (tolerance-interval) acceptance limits for two compendial tests: USP <905> Content Uniformity and USP <711> Dissolution.
- The original system is a SAS/AF application — reliable statistically, but built on a platform that is expensive to license, hard to install, and difficult to extend.
- The underlying methodology is sound. The problem was never the math — it was everything around it.

The SAS Pain Points

- ! **Aging platform**
SAS/AF interface, hard to run on modern systems
- ! **Licensing cost**
Ongoing SAS licensing just to run one tool
- ! **Hard to maintain**
Legacy macro code, few people can safely edit it
- ! **Limited access**
Not easy to share, install, or extend across teams

Four Modules, One Shared Engine



CUSP1

Content Uniformity

Single location / composite sample



CUSP2

Content Uniformity

Multiple locations — variance components



DISP1

Dissolution

Single location



DISP2

Dissolution

Multiple locations — variance components

Every module supports the same three analysis modes:

Acceptance-Limit Table

Probability of Passing

Sample Probability

Built on Rigorous, Published Statistics

- Every formula traces back to classical sampling theory — starting from Cochran's theorem: the sample mean and sample variance are independent and each follows a known distribution.
- This is the same family of methods as ASTM E2709 / E2810 — parametric tolerance intervals for compendial testing.
- Bonferroni-style inequalities keep every result a guaranteed conservative bound, not just a point estimate — that's why the code calls it “OVERBD,” not “probability.”



A full step-by-step derivation, with proofs, lives in Appendix E of the Validation Protocol.

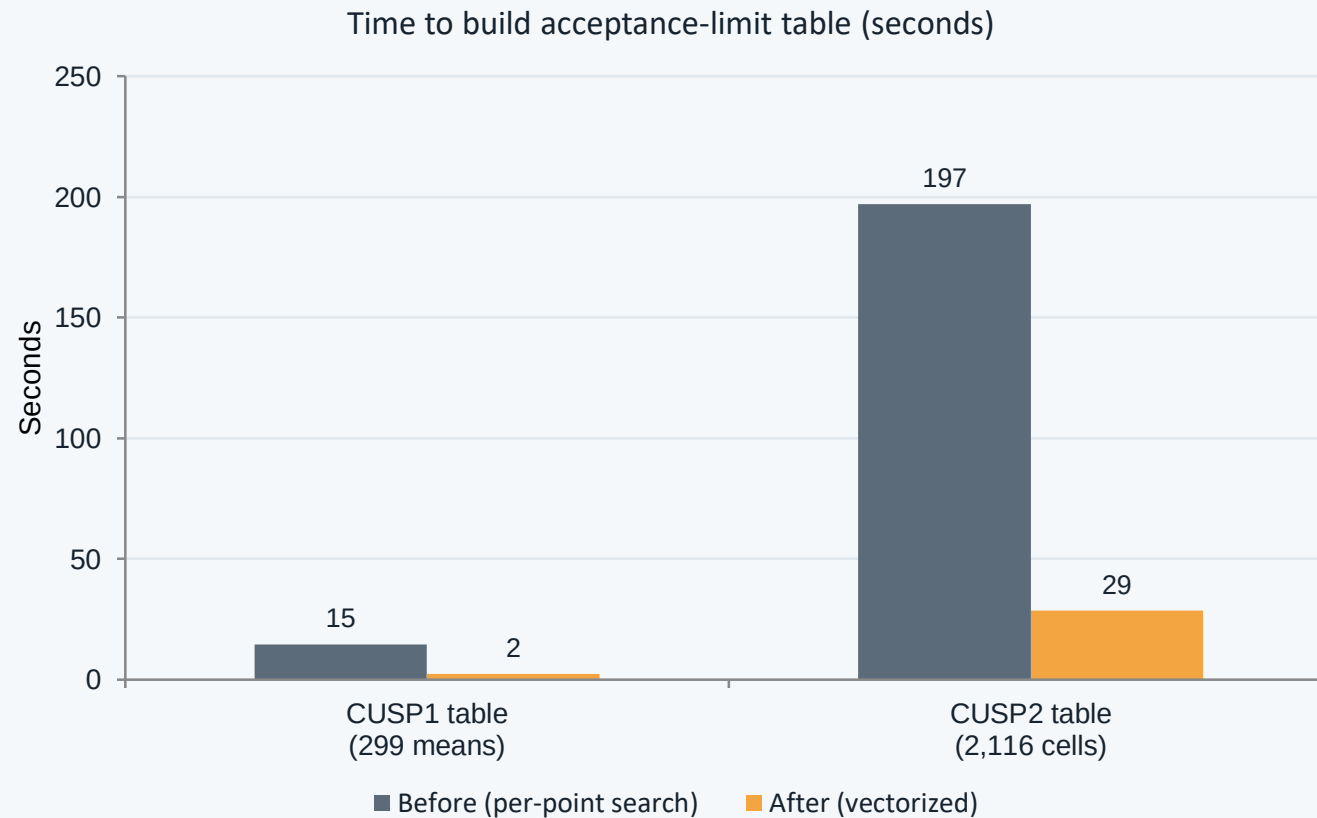
Line-by-Line Verification First

- Step one was never speed — it was fidelity: every SAS macro formula was translated and checked term-by-term before anything else changed.
- Only once the math was confirmed identical did any optimization work begin.
- The result: the same statistical answers the original system has produced for years, on a platform anyone can install with one command.

SAS Macro → Python Function

```
clcalc / cullu / cuulu  
content_uniformity_bound()  
  
COMPUTE  
dissolution_bound()  
  
CALCUSP1 / CALCUSP2  
acceptance_limit_table()  
  
SMPCUSP1 / SMPDISP1  
sample_probability()
```

Faster, Not Just Modern



What Changed

- Every probability formula rewritten to evaluate whole grids at once with numpy, instead of one point at a time.
- A vectorized bisection solves every grid point's root simultaneously, replacing SAS's original step-by-step search.
- Same formulas, same answers — just computed as arrays instead of loops.

The CUSP2 Discrepancy Investigation

In one narrow condition — very small variability combined with a large sample size — the Python result differed slightly from the original SAS output. Rather than dismiss it, we ran it to ground.



Documented in full as a standalone technical note, shared alongside the Validation Protocol.

Double-Checked in a Second Language

A separate R implementation of the CUSP1 and CUSP2 math was built independently, then run against the exact same shared test data set — giving us agreement from a language and codebase with no shared bugs.

- Same formulas re-derived independently in R, documented for peer review
- Run against every case in the shared Appendix G test data set
- Every result agreed with PyCuDAL within numerical tolerance

86 / 86

test cases matched

CUSP1 + CUSP2, across the full shared test data set

A Real Validation Package

Appendices

- A — Program Listing
- B — Primary Windows
- C — Default Output for Analysis (R Code)
- D — Navigation and Error Checks
- E — Math / Lower Bound Calculations
- F — Program Description
- G — Program Test Data Sets
- H — Curriculum Vitae's

Forms

- 1 — Load and Run Program
- 2 — Primary Window Navigation
- 3 — Program Strategy / Python Code Verification
- 4 — Test Data Agreement
- 5 — Problem / Request Report

Plus: a full User Manual, and a standalone Annexure covering the CUSP2 investigation.

Where Things Stand Today

Done

- ✓ Core translation complete, verified against SAS formula-by-formula
- ✓ Performance-optimized — fully vectorized acceptance-limit search
- ✓ CUSP2 edge case investigated and transparently documented
- ✓ Independent R cross-verification complete (86/86 cases)
- ✓ Validation Protocol drafted — all appendices and forms structured

In Progress / Next

- Community-driven validation execution (Forms 1, 2, 4)
- Broader accessibility — GUI exploration

Built in the Open

- Published on GitHub, fully open-source, currently at release v1.0.8
- Shared with the ISPE Open Forum community, requesting help completing validation
- Posted for visibility on LinkedIn — open to anyone who wants to try it or contribute

Get Involved

- 1 Try it — run real or sample cases
- 2 Help execute the validation forms
- 3 Report anything unexpected

Repository: github.com/moazelessawey/pycudal

Latest release: github.com/moazelessawey/pycudal/releases/tag/v1.0.8

What's Next

Now

Community-driven validation —
forms, test execution, issue
tracking



Next

Expand test data coverage across
edge cases



Then

Explore a GUI for non-technical
users



Later

Formal release once validation
closes

Let's Build This Together

Try it. Test it. Help validate it.

Repository:

github.com/moazelessawey/pycudal

Latest release:

github.com/moazelessawey/pycudal/releases/tag/v1.0.8

Thank you for watching.